

FathomNet 2026 — Backup & Extras Plan v4

When to use: After your master plan is executing on schedule AND you have time or budget remaining. Each item is optional polish.

Budget for extras: Up to ~\$66 of master plan headroom plus whatever compute you have left. Most items are nearly free.

Changelog from v3: - Added reference to master plan v5's new "Data Gotchas" section — several backup items have context there (especially the resolution shift for distribution analysis item #4). - Item #4 (distribution shift analysis) now has pre-known findings to start from.

Changelog from v2: - **Per-class NMS IoU tuning promoted to master plan** (Phase 7). It was too impactful under mAP@[.50:.95] to leave as backup. - **Score calibration now in master plan** (isotonic + temperature). Was partially covered in v2 — now explicit in Phase 7. - Reranked priorities to reflect the strict metric (localization-improving items bumped up).

How to read each item: - Problem it solves - Design principle (what it actually does) - Decision criteria (when to include vs skip) - Expected gain and cost - Light implementation reference

Priority Ranking (revised for mAP@[.50:.95])

If you can only do some, do them in this order:

1. **Per-class confidence thresholds** (2h, \$0) — direct score gain
2. **Seed ensemble** (1 day, ~\$24) — more ensemble diversity
3. **Active learning loop** — for LLNL career narrative, not leaderboard
4. **Distribution shift analysis** (4h, \$0) — NEW — understand val/LB gap
5. **Everything else** — nice-to-haves, only if well ahead

Items 1 and 4 combined can add +0.5 to 1.5 mAP@[.50:.95] with effectively zero GPU cost. Do them first if master plan completed smoothly.

1. Per-class confidence thresholds (2 hours, \$0)

Problem

mAP is computed independently per class. The optimal confidence threshold for "bony fish" (common, precision-dominated) differs from the optimal threshold for "octopus" (rare, recall-dominated). Default detectors use one global threshold for all classes.

Design principle

Grid search the confidence threshold independently per class on val data. Lock per-class thresholds at inference time.

Decision criteria

```
IF class-count imbalance ratio > 10:  
  INCLUDE — per-class tuning matters more when classes behave differently  
IF per-class AP variance is high (some classes >0.6, others <0.2):  
  INCLUDE  
ELWAYS:  
  INCLUDE — it's nearly free
```

Effectively: always do it.

Expected gain

+0.5 to 1 mAP@[.50:.95].

Methodology

1. Collect OOF (out-of-fold) predictions from Phase 6's 5-fold CV
2. For each class, sweep threshold from 0.05 to 0.95 in steps of 0.05
3. Compute per-class AP at each threshold (at each IoU threshold — they may differ per class!)
4. Lock the threshold that maximized that class's contribution to mAP@[.50:.95]
5. Apply all 32 thresholds at inference time: `if pred.score < thresholds[pred.class]: drop`

Light code reference

```

for cls_id in range(32):
    best_map, best_thresh = 0, 0.25
    for t in np.arange(0.05, 0.95, 0.05):
        # Compute this class's contribution to overall mAP@.50:.95
        mAP_contribution = compute_class_ap_at_thresholds(preds, gt, cls_id, threshold=t)
        if mAP_contribution > best_map:
            best_map, best_thresh = mAP_contribution, t
    thresholds[cls_id] = best_thresh

```

Failure mode

Overfitting to val data. Check that improvement on held-out fold is similar to improvement on fit fold. If they diverge, use coarser grid (0.1 steps instead of 0.05).

2. Seed ensemble (1 day, ~\$24)

Problem

Your 5-fold CV ensemble has model diversity from *data* splits, and architectural diversity from *RT-DETR*. You have no diversity from *initialization randomness*.

Design principle

Train 2 additional YOLOv11m models on all data with different random seeds (same architecture, same data, different init). Adds 2 more ensemble members with decorrelated errors purely from stochastic training dynamics.

Decision criteria

```

IF Phase 6 completed on schedule AND GPU budget > $30:
    INCLUDE seed ensemble
IF ensembling 5-fold alone gives > 1 mAP over best single fold:
    Ensembling is helping – more diversity will help more
    INCLUDE
IF ensemble already >2 points better than best single:
    Diminishing returns likely – SKIP

```

Expected gain

+0.3 to 0.5 mAP@.50:.95]. WBF is what squeezes value here — seed variance produces different box coordinates, which WBF averages into tighter final boxes.

Methodology

```

for seed in [1337, 2024]:
    torch.manual_seed(seed)
    np.random.seed(seed)
    # Otherwise identical to Phase 6 training config
    train(model, data_all, seed=seed)

```

Adds 2 members -> 8-model ensemble. Re-tune ensemble weights after adding.

3. Active learning loop (5 days, ~\$8)

Most impressive differentiator for LLNL/MBARI narrative. Marginal for competition leaderboard.

Problem

You'd like more labeled marine data, but annotation is expensive. Which images from the 400k unlabeled FathomNet DB should you manually label to gain the most?

Design principle

Use model uncertainty to prioritize. For each unlabeled image: - Run inference N times with dropout enabled (Monte Carlo Dropout) - Measure prediction variance across passes - High variance = model is uncertain = this image would be valuable to label

Label top-K uncertain images. Retrain. Uncertainty should decrease globally.

Decision criteria

```
IF you're specifically building your LLNL internship narrative:
    INCLUDE — this is the "how to efficiently label data" problem both MBARI and NIF face
IF > 3 days remaining after master plan:
    INCLUDE — useful learning
ELSE:
    SKIP — marginal leaderboard impact, not worth the time otherwise
```

Constraint to check

Competition rules forbid adding new *labeled* marine images. So the active learning loop can't directly produce more training data for your competition submission. It can, however, be written up as a methodology demonstration for your career narrative ("here's how I'd efficiently label if I were at MBARI"). Don't try to sneak the extra labels into the submission.

Expected gain

- Competition: marginal (constraint above).
- Career/learning: significant. Direct application to LLNL internship.

Methodology

1. Enable dropout at inference time: `model.model.train()` (switches dropout to training mode, rest of model still frozen)
2. Run `predict()` 10 times on each candidate image
3. Compute variance in detection counts, class distribution, or box coordinates
4. Rank images by variance
5. Report results as a standalone analysis notebook — **not** integrated into competition training

Light code reference

```
def mc_dropout_uncertainty(model, image, n=10):
    model.model.train() # enable dropout
    counts = [len(model.predict(image, conf=0.1)[0].boxes) for _ in range(n)]
    model.model.eval()
    return np.var(counts) # higher = more uncertain
```

Writeup angle for LLNL

"Implemented active learning loop demonstrating that annotation burden could be reduced by ~X% while matching full-dataset performance. Directly applicable to NIF data labeling bottleneck." Frame as methodology, not as a competition submission component.

4. Distribution shift analysis (4 hours, \$0) — NEW

Problem

Under `mAP@[.50:.95]`, you're especially vulnerable to train/test distribution differences — and the 2025 post-mortem confirms FathomNet has this problem. If your val mAP is consistently higher than leaderboard mAP, you need to understand why.

Pre-known findings (from master plan Data Gotchas section)

Already confirmed from JSON inspection: - **Test resolution distribution is heavily skewed vs train.** Train: 53% 1920×1080, 47% 4K. Test: 83% 1920×1080, **14% 720×486**, plus small stragglers (720×366, 640×486, etc). - **Temporal range of training spans 6+ years** (2017–2023). Test dates are stripped — can't verify temporal alignment.

The distribution shift hypothesis is thus pre-confirmed. What this analysis adds is *quantifying per-class and per-resolution impact*.

Design principle

Compute distribution statistics on your val set and compare to patterns you see on submission-scale inference. Identify which axes of variation are under-represented in training, and which classes get hit hardest by the 720×486 subset.

What to compute

1. **Per-resolution mAP** — split test images by resolution, submit separately, compare. If 720×486 mAP is much worse than 1920×1080 mAP, SAHI slice tuning is the fix.
2. **Size distribution of detections** — are test objects genuinely smaller than train? (already partially answered — yes, for the 720×486 subset)
3. **Density distribution** — are test images more crowded than train?
4. **Per-class difficulty** — which classes have biggest val-vs-LB gap?
5. **False positive characterization** — are they mostly small, edge, or low-confidence?

Decision criteria

```
ALWAYS DO IT after first 2-3 submissions.  
IF val-LB mAP gap > 3 points:  
    REQUIRED — you need to understand why before tuning further  
IF val-LB gap < 1 point:  
    Optional, but useful for final push
```

Expected gain

Indirect. Doesn't improve models directly, but tells you *which* of the above techniques to invest more in. Often saves hours of wasted experimentation.

Methodology

1. Split test images by resolution buckets: 1920×1080, 720×486, 4K, other
2. Run best model on each subset separately, record mAP per subset
3. If 720×486 mAP << 1920×1080 mAP -> tune SAHI slice size for low-res images (try 240, 320, 400)
4. If 4K mAP << 1920×1080 mAP -> large-image failure; try larger slices or downsample first
5. Plot predicted box sizes vs train box sizes — histograms overlaid
6. If predicted size distribution skews smaller -> invest more in SAHI and multi-scale
7. If predicted density is higher -> invest more in per-class NMS

5. IoU estimation branch for pseudo-label filtering (3 days, ~\$10)

Problem

Plain pseudo-labeling filters by confidence. But a high-confidence box may still be spatially misaligned — particularly costly under $mAP@[.50:.95]$. You'd rather filter by *expected IoU with the true box* — but you don't know the true box.

Design principle

Train an auxiliary head on labeled data that predicts IoU between a detection and ground truth. At inference on unlabeled data, this predicted IoU becomes a quality filter.

Decision criteria

```
IF Soft Teacher gave positive but marginal gain (<1 mAP):  
    INCLUDE IoU predictor to improve pseudo-label quality  
IF plain pseudo-labeling already works well:  
    SKIP — marginal additional gain
```

Expected gain

+1 to 2 $mAP@[.50:.95]$ over plain pseudo-labeling / Soft Teacher when PU severity is high. Higher gain under strict metric because IoU quality is directly what the metric measures.

Methodology

1. Add IoU regression head parallel to classification/regression heads
2. Loss: predict IoU between each proposal and its matched GT box during supervised training
3. At inference on unlabeled data, predicted IoU threshold (e.g., > 0.7) replaces confidence threshold for pseudo-labels

6. Knowledge distillation (2 days, ~\$16)

Problem

Large models are accurate but slow. Could distill their “dark knowledge” into a smaller model that runs faster — useful if inference time matters.

Design principle

Train small model (student) to match large model's (teacher) soft predictions, not just hard labels. Soft predictions contain information about uncertainty and class relationships.

Decision criteria

```
IF inference time is constrained (it's not, for this competition):
    Useful
IF you want an additional ensemble member at a different scale:
    Mildly useful
ELSE:
    SKIP — marginal value for this competition
```

Expected gain

+0.5 to 1 mAP on distilled smaller model. Also useful as ensemble diversity.

Methodology

```
teacher_logits = teacher(x)          # frozen, larger model
student_logits = student(x)          # trained
distill_loss = KL_div(
    softmax(student_logits / T),
    softmax(teacher_logits / T)
) * T2
hard_loss = cross_entropy(student_logits, labels)
total =  $\alpha$  * distill_loss + (1 -  $\alpha$ ) * hard_loss
#  $\alpha$  = 0.7, T = 4 are typical
```

7. Feature-level augmentation (FASA) (2 days, ~\$4)

Problem

Rare classes with <50 instances can't benefit much from pixel-level augmentation — you still have the same underlying visual variety.

Design principle

Augment in *feature space* instead. Sample new feature vectors for rare classes from the statistics of their labeled examples.

Decision criteria

```
IF EDA shows any class with < 30 training instances:
    INCLUDE — pixel-level augmentation is hitting a ceiling
IF RFS + class-aware copy-paste already helped rare classes substantially:
    SKIP — diminishing returns
```

Expected gain

+0.3 to 0.5 mAP on rare classes only.

Methodology

1. Extract features for all rare-class instances using your trained backbone
2. Compute per-class feature mean and std
3. Generate synthetic features: $\text{synthetic} = \mu + \sigma \times \epsilon$, where $\epsilon \sim N(0, 0.5)$
4. Inject into training batch alongside real features during classifier training

8. Multi-task image-level classification head (2 days, ~\$16)

Problem

The PU problem means many objects go unlabeled. But at the *image level*, labels are more reliable — if any bony fish is labeled in an image, the image definitely contains bony fish.

Design principle

Add an auxiliary head predicting the set of categories present in the image. Multi-label binary cross-entropy. The auxiliary signal helps the backbone learn better features without requiring complete bounding box annotations.

Decision criteria

```
IF PU severity is extreme (>60% single-category labels):  
  INCLUDE — image-level signal is more reliable than box-level  
IF Phase 3 differentiators already gave good gains:  
  OPTIONAL — incremental benefit
```

Expected gain

+0.5 to 1 mAP.

Methodology

```
backbone_features = backbone(x)  
detections = detection_head(backbone_features)  
image_labels = image_classifier_head(pool(backbone_features[-1]))  
  
total_loss = detection_loss + 0.3 × image_multilabel_bce_loss
```

Particularly valuable for PU because it leverages a more reliable supervision signal.

9. DINOv2 ViT-L/14 backbone (4 days, ~\$60)

Problem

Maybe a bigger backbone helps. DINOv2 ViT-L has ~300M parameters vs ResNet50's 25M. Better features, potentially.

Decision criteria

```
IF Phase 6 mAP has plateaued AND nothing in master plan is left to tune:  
  MAYBE include — significant integration effort  
IF time is tight OR simpler ideas haven't been exhausted:  
  SKIP — ROI is low, risk of implementation bugs is high
```

Expected gain

+1 to 2 mAP if integration succeeds. Zero if integration has bugs.

Cost

High engineering effort. YOLO expects CNN backbone by default; integrating a ViT backbone requires custom feature pyramid construction. Plan for 3–4 days of integration + debugging even before training.

Reality check

Not recommended unless you have a full week left. Better to use BioClip2 (which is already trained on marine data) than to struggle with ViT-L integration.

10. Mosaic + CutMix additions (half day, \$0)

Problem

Default Ultralytics mosaic + mixup helps. Could add more augmentation variety.

Design principle

- **Gradual augmentation decay** — strong early, weak late
- **CutMix** — cut a patch from one image, paste into another, blend labels proportionally
- Combine with `close_mosaic=20` (already in master plan)

Decision criteria

```
IF val mAP curve shows overfitting (train << val gap grows):  
  INCLUDE — more regularization helps  
ELSE:  
  SKIP — added complexity for marginal gain
```

Expected gain

+0.2 to 0.5 mAP.

11. Working note paper submission (3-5 days of writing)

What it is

Optional 4–6 page paper describing your method. Submitted to CEUR-WS proceedings via the CLEF 2026 Lab.

Why consider it

- Gets you into the officially published CLEF ranking (not just Kaggle leaderboard)
- Publishable output — listable on resume, LinkedIn
- Strong conversation piece for MBARI/LLNL internship applications

Decision criteria

IF final leaderboard position is top 15%:
STRONGLY CONSIDER writing the paper

IF position is top 30% AND LLNL mentor encourages it:
DO IT — you have a 3-week window from May 7 to May 28

IF position is mediocre:
SKIP — not worth the writing time

Paper structure (if doing it)

- **Method** — describe the differentiator stack: SSL pretraining, Kiryo PU + EFL, Soft Teacher, hierarchical loss, SAHI + WBF ensemble + per-class NMS + calibration
- **Results** — official leaderboard result plus ablation table showing each differentiator’s contribution
- **Analysis** — per-class AP, failure mode inspection
- **Key tables** — ablations are the most cite-able content

Timeline

- **May 7** — Kaggle deadline, final submission known
- **May 8-27** — writing window (~3 weeks)
- **May 28** — CLEF working note deadline

Compute Budget for Extras

Item	Compute	Cost
1 (per-class confidence)	CPU only	\$0
2 (seed ensemble)	A100 40GB, ~12h	~\$24
3 (active learning)	RTX 4090, ~10h	~\$8
4 (distribution shift analysis)	CPU only	\$0
5 (IoU predictor)	A100 40GB, ~5h	~\$10
6 (distillation)	A100 40GB, ~8h	~\$16
7 (FASA)	RTX 4090, ~5h	~\$4
8 (multi-task head)	A100 40GB, ~8h	~\$16
9 (DINOv2-L)	A100 80GB, ~25h	~\$60
10 (CutMix)	RTX 4090, ~3h	~\$2
11 (paper)	—	\$0 (time only)

Items 1, 4 alone: \$0. Do these first — pure analysis gain.

Items 1-4: ~\$32, fits easily in the \$66 master plan headroom.

All items 1-8, 10: ~\$80, pushes total to ~\$310. Skip #6 or #8 if strictly capping at \$300.

Quick Decision Tree

After Phase 7 submission:

IF you have > 3 days left before deadline:

- # Free analysis first

- DO items 1, 4 (per-class confidence + distribution shift analysis)

- IF items 1 + 4 revealed specific weaknesses:

 - Pick targeted extras from 5-8 to address them

- IF additional leaderboard push wanted:

 - DO item 2 (seed ensemble)

- IF building LLNL narrative is a priority:

 - DO item 3 (active learning)

IF you have 1-2 days left:

- DO only items 1 and 4 (per-class confidence + shift analysis)

- ~6 hours of work, ~\$0 cost, +0.5-1.5 mAP typically

IF you have hours only:

- DO only item 1 (per-class confidence)

- 2 hours of work, +0.5 to 1 mAP

IF you're behind schedule:

- Don't touch backup plan

- Polish your master plan submission instead